NAME:U.Prabhavathi **COURSE CODE:DSA0107**

REG NO:192472067 **COURSE NAME:C++**

UNIT IV
INHERITANCE & POINTERS

Answers – Debugging and Pseudo-Code Formulation

SECTION A – DEBUGGING QUESTIONS

Question 1: Diamond Problem in Multipath Hybrid Inheritance

1. Compilation error: the expression ta.idNumber is ambiguous because TeachingAssistant contains two Person subobjects—one through Student and another through Employee.

2. Memory layout (conceptual): TeachingAssistant → Student → Person(idNumber = 101), and TeachingAssistant → Employee → Person(idNumber = 202). The compiler cannot decide which idNumber is intended.

3. Correction: Student and Employee inherit Person virtually. The most-derived class TeachingAssistant initializes the single shared Person subobject.

```
#include <iostream>
using namespace std;

class Person {
public:
   int idNumber;
   Person(int id = 0) : idNumber(id) {
      cout << "Person initialized: " << idNumber << '\n';
   }
};
class Student : virtual public Person {
public:
   Student(int id) : Person(id) {}
};
class Employee : virtual public Person {
public:
   Employee(int id) : Person(id) {}
};
class TeachingAssistant : public Student, public Employee {
public:
   TeachingAssistant(int id)
      : Person(id), Student(id), Employee(id) {}
};
```

```
int main() {
    TeachingAssistant ta(101);
    cout << "TA ID: " << ta.idNumber << '\n';
    return 0;
}
```

4. Constructor rule: for a virtual base class, the most-derived class is responsible for constructing it. Therefore TeachingAssistant directly calls Person(id). The Person calls written in Student and Employee are ignored when a TeachingAssistant is created.

Console output preview:



Question 2: Missing Base Constructor in Derived Initializer List

1. Device has only a parameterized constructor. Since Sensor does not explicitly call Device(mac), the compiler tries to call Device() automatically. No default constructor exists, so compilation fails before the Sensor constructor body runs.

2. Constructor order: base class Device first; then Sensor's members; then the Sensor constructor body. Base-class construction always precedes the derived-class body.

```
#include <iostream>
#include <string>
using namespace std;

class Device {
protected:
    string macAddress;
public:
    Device(string mac) : macAddress(mac) {
        cout << "Device configured with MAC: " << macAddress << '\n';
    }
};
class Sensor : public Device {
private:
    int samplingRate;
public:
    Sensor(string mac, int rate)
        : Device(mac), samplingRate(rate) {}
    void display() const {
        cout << "MAC: " << macAddress
             << ", Rate: " << samplingRate << " Hz\n";
```

```
    }
};
int main() {
    Sensor tempSensor("00:1A:2B:3C:4D:5E", 100);
    tempSensor.display();
    return 0;
}
```

3. The initializer list Device(mac) passes the required MAC address to the base constructor. samplingRate is also initialized in the list.

Console output preview:



Question 3: Base Pointer Member Access and Object Slicing

1. vPtr has the static type Vehicle*. C++ checks member access using the pointer's declared type, so engageTurbo() is unavailable through Vehicle*. The fact that the address currently refers to a SportsCar does not add derived members to the Vehicle interface.

2. Object slicing occurs when a SportsCar is copied into a Vehicle object by value. Only the Vehicle part is copied; turboEnabled is not part of the resulting Vehicle object.

3. A checked downcast using dynamic_cast requires a polymorphic base, so the corrected example adds a virtual destructor. A reference to the derived object avoids slicing.

```
#include <iostream>
using namespace std;

class Vehicle {
public:
    int maxSpeed;
    Vehicle(int s) : maxSpeed(s) {}
    virtual ~Vehicle() = default;
    void honk() const { cout << "General Horn\n"; }
};
class SportsCar : public Vehicle {
public:
    bool turboEnabled;
    SportsCar(int s, bool turbo) : Vehicle(s), turboEnabled(turbo) {}
    void engageTurbo() const { cout << "Turbo Engaged!\n"; }
};
int main() {
```

```
    Vehicle* vPtr = new SportsCar(300, true);
    vPtr->honk();

    SportsCar* sportsPtr = dynamic_cast<SportsCar*>(vPtr);
    if (sportsPtr != nullptr) sportsPtr->engageTurbo();

    SportsCar car(250, true);
    Vehicle& vehicleRef = car; // reference: no slicing
    cout << "Speed through base reference: " << vehicleRef.maxSpeed << '\n';
    cout << "Turbo retained in derived object: "
         << boolalpha << car.turboEnabled << '\n';

    delete vPtr;
    return 0;
}
```

Console output preview:



Question 4: Incorrect Return Type in this Pointer Method Chaining

1. setWidth() returns WindowBox*. The result is a pointer, so the next operation must use -> rather than the dot operator. The expression .setHeight() is therefore invalid.

2. this is a pointer to the current object (WindowBox*). *this is the current object itself as an lvalue; returning WindowBox& allows chained dot calls.

```
#include <iostream>
using namespace std;

class WindowBox {
private:
    int width, height;
public:
    WindowBox() : width(0), height(0) {}
    WindowBox& setWidth(int width) {
        this->width = width;
        return *this;
    }
    WindowBox& setHeight(int height) {
        this->height = height;
        return *this;
```

```
    }
    void display() const {
        cout << "Window: " << width << " x " << height << '\n';
    }
};
int main() {
    WindowBox box;
    box.setWidth(800).setHeight(600);
    box.display();
    return 0;
}
```

Console output preview:



Question 5: Private Inheritance Access Violations

1. With private inheritance, public and protected members of RawSocket become private members of SecureChannel. SecureChannel can call openPort() internally, but main() cannot call closePort() through a SecureChannel object.

2. Upcasting is inaccessible because the RawSocket base relationship is private. Public upcasting is allowed when the base class is publicly accessible.

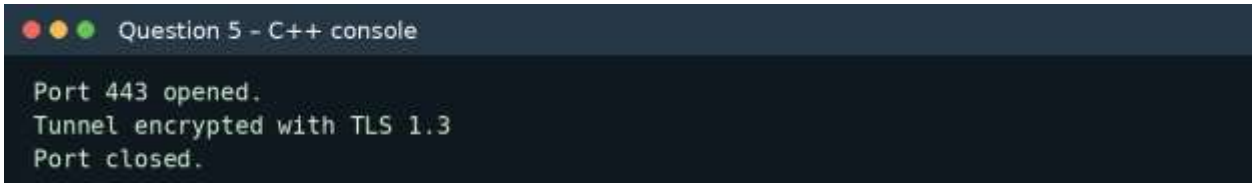
3. Approach A uses a public using-declaration to expose the selected base member. This preserves private inheritance while making closePort() public through SecureChannel.

```
#include <iostream>
using namespace std;

class RawSocket {
public:
    void openPort(int port) {
        cout << "Port " << port << " opened.\n";
    }
    void closePort() {
        cout << "Port closed.\n";
    }
};
class SecureChannel : private RawSocket {
public:
    using RawSocket::closePort;
    void establishEncryptedTunnel(int port) {
```

```
        openPort(port);
        cout << "Tunnel encrypted with TLS 1.3\n";
    }
};
int main() {
    SecureChannel channel;
    channel.establishEncryptedTunnel(443);
    channel.closePort();
    return 0;
}
```

Console output preview:



SECTION B – PSEUDO-CODE WRITING QUESTIONS

The following solutions use the structured pseudo-code conventions specified in the question bank: CLASS, INHERITS, VIRTUAL, ABSTRACT CLASS, ABSTRACT METHOD, NEW, POINTER TO, and RETURN REF THIS.

Question 1: University ERP Hybrid Hierarchy with Virtual Base Classes

Person is the common virtual base. Student and Faculty inherit it virtually, so TeachingAssistant contains one Person record and can access its fields without ambiguity.

```
CLASS Person
    ATTRIBUTES: personID, fullName, email
    CONSTRUCTOR(id, name, mail)
        personID <- id
        fullName <- name
        email <- mail
END CLASS

CLASS Student INHERITS VIRTUAL Person
    ATTRIBUTES: gpa, creditsEarned
    CONSTRUCTOR(id, name, mail, g, credits)
        BASE(id, name, mail)
        gpa <- g
        creditsEarned <- credits
END CLASS

CLASS Faculty INHERITS VIRTUAL Person
    ATTRIBUTES: academicDepartment, rankTitle
    CONSTRUCTOR(id, name, mail, dept, rank)
        BASE(id, name, mail)
        academicDepartment <- dept
        rankTitle <- rank
END CLASS

CLASS TeachingAssistant INHERITS Student, Faculty
    ATTRIBUTES: stipendAmount, assignedCourse
    CONSTRUCTOR(id, name, mail, g, credits, dept, rank, stipend, course)
        INITIALIZE VIRTUAL BASE Person(id, name, mail)
        INITIALIZE Student(...)
        INITIALIZE Faculty(...)
        stipendAmount <- stipend
        assignedCourse <- course
    METHOD DisplayTARecord()
        OUTPUT personID, fullName, gpa, academicDepartment, stipendAmount
END CLASS
```

```
MAIN
   CREATE TeachingAssistant TA("U101", "Anu", "anu@college.edu",
                               8.7, 90, "CSE", "Assistant", 12000, "C++")
   TA.DisplayTARecord()
END MAIN
```

Field	Sample value
Person ID	U101
Name	Anu
GPA	8.7
Department	CSE
Stipend	12000

Console output preview:



Question 2: Polymorphic Payroll with Abstract Base Class and Base Pointers
Employee is abstract because CalculatePay() is a pure virtual method. SalariedEmployee and HourlyEmployee implement it. The array stores Employee pointers, allowing the correct derived override to run for each object.

```
ABSTRACT CLASS Employee
   PROTECTED: employeeID, name
   CONSTRUCTOR(id, employeeName)
      employeeID <- id
      name <- employeeName
   ABSTRACT METHOD CalculatePay() RETURNS REAL
   METHOD DisplayHeader()
      OUTPUT employeeID, name
END CLASS

CLASS SalariedEmployee INHERITS Employee
   ATTRIBUTE: monthlySalary
   METHOD CalculatePay()
      RETURN monthlySalary
END CLASS

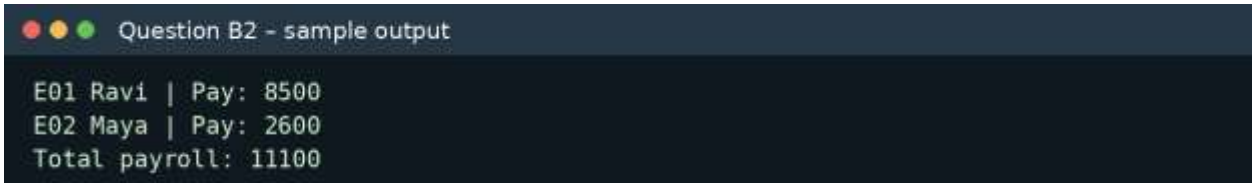
CLASS HourlyEmployee INHERITS Employee
```

```
   ATTRIBUTES: hourlyWage, hoursWorked
   METHOD CalculatePay()
      IF hoursWorked <= 40 THEN
         RETURN hourlyWage * hoursWorked
      ELSE
         RETURN (hourlyWage * 40) +
               (hourlyWage * 1.5 * (hoursWorked - 40))
      END IF
END CLASS

MAIN
   DECLARE employees AS ARRAY OF POINTER TO Employee
   employees[0] <- NEW SalariedEmployee("E01", "Ravi", 8500)
   employees[1] <- NEW HourlyEmployee("E02", "Maya", 50, 48)
   total <- 0
   FOR EACH employee IN employees
      employee->DisplayHeader()
      pay <- employee->CalculatePay()
      OUTPUT pay
      total <- total + pay
   END FOR
   OUTPUT total
   DELETE all employees
END MAIN
```

Employee	Calculation	Pay
Salaried	8500 per month	8500
Hourly	(50 × 40) + (50 × 1.5 × 8)	2600
Total	8500 + 2600	11100

Console output preview:



Question 3: Composite Canvas Engine with Member Classes and this Chaining

Circle has a Point2D object as a member. This is composition, also called a has-a relationship. Both setter methods return a reference to the current Circle, enabling chained calls.

```
CLASS Point2D
   PRIVATE: x, y
   CONSTRUCTOR(newX = 0.0, newY = 0.0)
      x <- newX
```

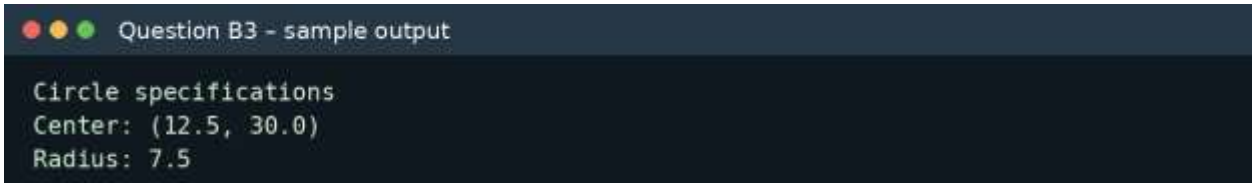
```
    y <- newY
  METHOD SetCoordinates(newX, newY)
    x <- newX
    y <- newY
  METHOD Display()
    OUTPUT x, y
END CLASS

CLASS Circle
  PRIVATE: centerPoint AS Point2D
           radius AS REAL
  CONSTRUCTOR()
    centerPoint <- NEW Point2D(0.0, 0.0)
    radius <- 1.0
  METHOD SetCenter(x, y) RETURNS REF Circle
    centerPoint.SetCoordinates(x, y)
    RETURN REF THIS
  METHOD SetRadius(r) RETURNS REF Circle
    IF r > 0 THEN radius <- r
    ELSE OUTPUT "Radius must be positive"
    RETURN REF THIS
  METHOD Draw()
    OUTPUT centerPoint, radius
END CLASS

MAIN
  DECLARE myCircle AS Circle
  myCircle.SetCenter(12.5, 30.0).SetRadius(7.5).Draw()
END MAIN
```

Property	Value
Center X	12.5
Center Y	30.0
Radius	7.5

Console output preview:



Question 4: Multi-Level Autonomous Fleet Telemetry with Base Pointers

The hierarchy is Vehicle → AutonomousVehicle → DeliveryDrone. Constructor arguments are passed upward using BASE(...) initializer delegation. A Vehicle pointer can access general vehicle features; a derived pointer is used for drone-specific telemetry.

```
CLASS Vehicle
   ATTRIBUTES: vehicleVIN, baseWeightKg
   CONSTRUCTOR(vin, weight)
      vehicleVIN <- vin
      baseWeightKg <- weight
   METHOD DisplaySpecs()
      OUTPUT vehicleVIN, baseWeightKg
END CLASS

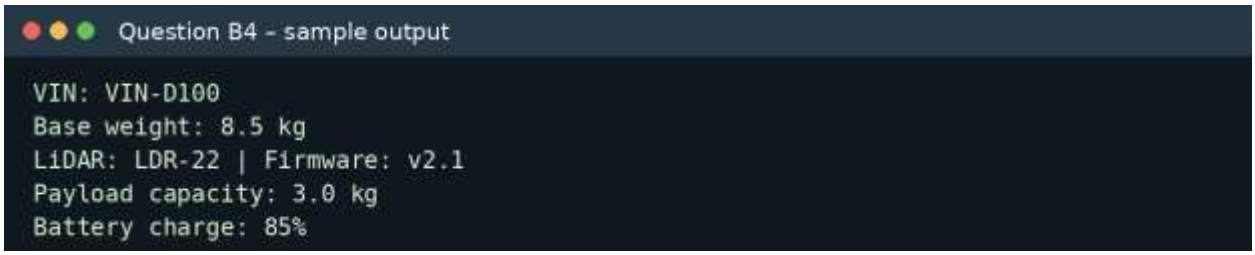
CLASS AutonomousVehicle INHERITS Vehicle
   ATTRIBUTES: lidarSensorID, firmwareVersion
   CONSTRUCTOR(vin, weight, lidar, firmware)
      BASE(vin, weight)
      lidarSensorID <- lidar
      firmwareVersion <- firmware
END CLASS

CLASS DeliveryDrone INHERITS AutonomousVehicle
   ATTRIBUTES: payloadCapacityKg, batteryChargePercent
   CONSTRUCTOR(vin, weight, lidar, firmware, capacity, battery)
      BASE(vin, weight, lidar, firmware)
      payloadCapacityKg <- capacity
      batteryChargePercent <- battery
   METHOD DisplayFlightStatus()
      DisplaySpecs()
      OUTPUT lidarSensorID, firmwareVersion
      OUTPUT payloadCapacityKg, batteryChargePercent
END CLASS

MAIN
   DECLARE dronePtr AS POINTER TO Vehicle
   dronePtr <- NEW DeliveryDrone("VIN-D100", 8.5, "LDR-22",
                                 "v2.1", 3.0, 85)
   dronePtr->DisplaySpecs()
   DECLARE fullPtr AS POINTER TO DeliveryDrone
   fullPtr <- DOWNCAST(dronePtr)
   fullPtr->DisplayFlightStatus()
   DELETE dronePtr
END MAIN
```

Constructor sequence	Order
Vehicle	1 – first
AutonomousVehicle	2 – second
DeliveryDrone	3 – last

Console output preview:



Question 5: Dynamic Object Pointer Array and Zero-Copy Sorting Engine

The catalog contains Product pointers. Bubble sort compares the prices of the pointed-to objects, but swaps only pointer variables. The Product objects themselves are not copied or moved.

```
CLASS Product
   ATTRIBUTES: sku AS STRING, price AS REAL, stockCount AS INTEGER
   CONSTRUCTOR(code, amount, stock)
      sku <- code
      price <- amount
      stockCount <- stock
   METHOD PrintProduct()
      OUTPUT sku, price, stockCount
END CLASS

MAIN
   DECLARE catalog[3] AS ARRAY OF POINTER TO Product
   catalog[0] <- NEW Product("Laptop", 1200, 5)
   catalog[1] <- NEW Product("Mouse", 25, 40)
   catalog[2] <- NEW Product("Monitor", 350, 12)

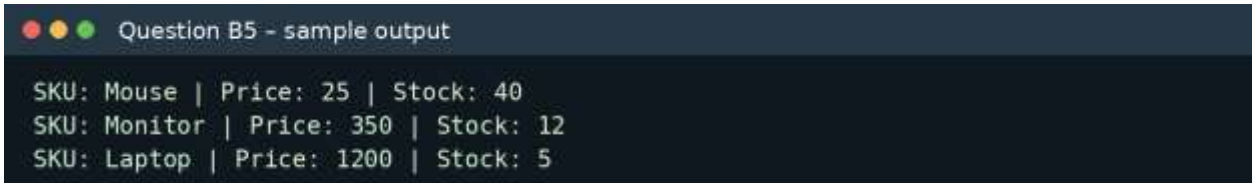
   FOR i <- 0 TO 1
      FOR j <- 0 TO 1-i
         IF catalog[j]->price > catalog[j+1]->price THEN
            temp <- catalog[j]
            catalog[j] <- catalog[j+1]
            catalog[j+1] <- temp
         END IF
      END FOR
   END FOR

   FOR EACH ptr IN catalog
```

```
    ptr->PrintProduct()
  END FOR
  FOR EACH ptr IN catalog
    DELETE ptr
  END FOR
END MAIN
```

Pass / comparison	Before	Action	After
Pass 1: Laptop 1200 vs Mouse 25	Laptop, Mouse, Monitor	Swap	Mouse, Laptop, Monitor
Pass 1: Laptop 1200 vs Monitor 350	Mouse, Laptop, Monitor	Swap	Mouse, Monitor, Laptop
Pass 2: Mouse 25 vs Monitor 350	Mouse, Monitor, Laptop	No swap	Mouse, Monitor, Laptop

Console output preview:



Final Notes

All five debugging examples show the corrected logic and expected console output. The pseudo-code section follows the question bank's required notation and includes representative dry-run values. Before submission, compile the five C++ programs in your required online compiler and replace the illustrative console previews with genuine screenshots if your instructor specifically requires website screenshots.

For online compilation, select C++ (C++17 or later), paste one corrected program at a time, click Run, and capture the output panel.